Structured Programming Methods

Published by:
NCC Education Limited
The Towers, Towers Business Park
Wilmslow Road, Didsbury
Manchester M20 2EZ
UK

Chapter 1 – History of Programming

Programming has developed gradually over the centuries, and there are several factors in human society that enabled programming to be developed. The roots of programming lie with the development of our numbering system in India and the Arab world. What can be seen as the first programs were written for machinery such as looms and calculating machines during the Industrial Revolution of the 18/19th centuries. It has taken many centuries for programming languages to reach their current sophistication, and the chapter aims to explore that history. The final part of the chapter introduces object-orientation, a feature of very recent languages, in more detail.

What is a Programming Language?

There are many definitions of what constitutes a programming language, and none of these is the 'correct' answer. What might have defined a programming language in the 19th century would not necessarily be detailed enough for a modern definition. This chapter aims to allow you to explore your own ideas of what you think a programming language might be. Programming languages are needed to allow human beings and computers to talk to each other. Computers, as yet, are unable to understand our everyday language or, in fact, the way we talk about the world. Computers understand logic expressed mathematically through what is known as machine code. Computer language consists of 1s and 0s or the binary system, which the majority of human beings would find very difficult to communicate in. Computer languages enable humans to write in a form that is more compatible with a human system of communication. This is then translated into a form that the computer can understand.

Here are some different ideas on what constitutes a programming language: -

- A programming language has been defined as a tool to help the programmer.
- A way of writing that can be read by both a human being and a machine.
- A sequence of instructions for the machine to carry out.
- A way for a human being to communicate with a machine that is unable to understand

natural language.

- A computer language offers a means of writing algorithms that can be understood by both human being and machine. Machines are unable to understand natural language, so a human being uses algorithms that are translated into machine code by the programming language. Machine code is difficult for humans to use, so a language 'translates' human readable language into machine readable form.
- A computer program offers humans a standard way of expressing algorithms to solve particular problems. As languages offer a convention it allows other humans to read the program, and change it if they need to.

Definition: Algorithm

A systematic procedure that produces, in a finite number of steps, the answer to a question or the solution of a problem.

Language Models

There are currently several kinds of languages: -

- Imperative languages are formed from collections of basic commands, most often assignments and Input/Output (I/O), where the execution is sequenced by control structures (e.g. loops, conditionals, blocks). Imperative (or procedural) languages specify explicit sequences of steps to follow to produce a result. These languages include C, Pascal, FORTRAN and Assembly.
- Functional languages are based on lambda-calculus from the 1930s. Programs consist of collections of function definitions and function applications. Higher order functions, abstract functions and lack of side effects might also be associated with functional programs. Examples include LISP, 'pure' Scheme, FP, ML and Haskell. Programs written in a functional language are generally compact and elegant, but have tended, until recently, to run slowly and require a lot of memory.
- Logic programming consists of collections of statements within a particular logic.

Most typically that logic is predicate logic. The original logic programming language was Prolog.

- Object-oriented (OO) languages. Programs consist of objects that interact with each other. Some also associate inheritance and polymorphism with OO languages. Examples are Simula, Smalltalk-80, Eiffel and Java.
- Declarative languages are collections of declarations. Many functional and logic languages are also declarative. You describe a pattern to be matched without writing the code to match the pattern. Some patterns would be difficult to write by hand. Declarative languages describe relationships between variables in terms of functions or inference rules and the language executor (interpreter or compiler) applies some fixed algorithm to these relations to produce a result.
- Scripting languages work in conjunction with a larger application, support ontrol of a variety of applications, are interpreted, or some combination thereof.
- Parallel languages are collections of processes (or sometimes structured ata) that communicate with each other; these include C* and Ada.

Evaluating Languages

Programming languages can be evaluated from a number of viewpoints, depending on either the programmer, the environment in which the programmer works or the standards of the organization. When developing software, a programmer should consider which language is most suitable to the task, rather than relying on a language with which they are familiar. You wouldn't want to use a spreadsheet to develop a database. All the features of the language need to be considered rather than just one particular feature. A wrong choice can mean that the software has to be re-written, which can be very frustrating and time-consuming. If you are going to develop a system it is important to choose your language according to the goals that you have in mind. For instance, you may want to teach object-oriented programming so you might choose Smalltalk, you might want to create interactive multimedia and use Java or business applications in COBOL. If you want to create a large-robust system you probably won't want to use something that you normally to use for rapid development.

There are a number of different ways that the programmer can think about the design of the system, from the top-down of structured programming to object-oriented design issues. Some languages are geared towards one particular style of design, whilst others incorporate many types. Each of these language paradigms enables the programmer to consider the problem from a different viewpoint.

There are a few basic questions that can be asked to help when making these decisions: -

1. How readable is the language, to humans? If parts of the program are going to be read or altered separately from the entire program is might be worth considering how legible they are going to be. It is also useful to consider the length of names allowed in the language, for instance an early form of FORTRAN allowed for only 6 characters. This can lead to clumsy abbreviations that are difficult to read. Statements such as GO TO, FOR, WHILE and LOOP have increased the readability of programs, and lead to neater programs. These statements also affect the syntax or grammar.
2. When it comes to writing the program, how easy is it to write the program in this particular language? A programming language that is easy to write in can make the process easier and faster. It may help to reduce mistakes. FOR loops and other types of statement allow the programmer to write much simpler code. This will save time and money, and also make the program smaller.
2. How reliable is the language? Not all languages create robust programs, and some help the programmer to avoid making errors. A program that is not robust can cause errors, and code can 'decay'. Any language that helps the programmer to avoid mistakes will make it easier to use.
2. How much would it cost to develop using a given language? Is the language expensive to use and to maintain? Programs may need to be updated or re-developed, and an expensive language may make this prohibitive.
2. How complicated is the syntax going to be? Syntax is an important consideration. Clarity and ease of understanding are important, as is a syntax that seems logical and sensible. Errors are very likely to occur where one area of syntax too closely

resembles another, and the program may prove difficult to debug. Some theorists reason that if it is difficult to write a program to parse the language, then it follows that it will be problematical for the programmer to get it right.

2. Does the language have standards? Languages that have standards for writing programs have greater readability; for instance Java has standards for naming, commenting and capitalization.

Definition: Syntax

Rules defining the legal sequences of symbolic elements in a language. The syntax rules define the form of various constructs in the language, but say nothing about the meaning of these constructs. Examples of constructs are: expressions, procedures, and programs (in the case of programming languages) and terms, well-formed formulas and sentences (in the case of logical languages).

Study Note

Approaches accepted by the academic community and industry: -

- FORTRAN is an early specialized language used by the science, mathematical and engineering communities both in academia and in industry.
- COBOL is still used in industry, and historically it has been the language of business and the mainframe.
- Pascal was developed as a teaching language, and is not viewed as a bona fide development language. With the increase in interest in object orientation, Pascal has declined as a teaching language and Smalltalk, or latterly Java, is often used as a teaching tool in its place.
- The most common scripting languages used for web development are ActiveX (especially when related to plug-ins), JavaScript and VBScript. New developments such as Active Server Pages (ASP) can use any language so long as it is specified. When ASP was developed it was proprietary for the Windows Operating Systems and dependent on existing ownership of software, but otherwise essentially free for developers. ASP is now widely used and software has been developed to make it

platform independent. Other database scripting languages include Cold Fusion, which is a little more difficult to implement.

Open source programming has grown with the growth of Linux. The Linux operating system is free and the code is available for other programmers. This concept of making code available to the programming community has spread to other programs such as Netscape.

Chronology of Programming

Computation has been in existence for a lot longer than the computer. Machines or devices were developed to either use these computations or for their calculation. Though these systems may seem primitive, modern computing still uses some of these early systems of computation. The complex systems of calculation often developed hundreds of years ago form the basis of modern computer languages

Developments in programming have also been dependent on the computer hardware itself. Until the invention of the PC, programmers expected to share time on a mainframe computer. The mainframe was generally a gigantic computer that was given a room to itself. Modern personal computers have allowed programmers to work on their own machines, whilst the development of networks has meant that sharing has become possible.

The Language Generations

The language generations span many decades, and begin with the development of machine code. Each generation adds new features and capabilities for the programmer to use. Languages are designed to create programs of a particular type, or to deal with particular problems. Modern languages have led to the development of completely different styles of programming involving the use of more human-like or natural language and re-usable pieces of code. This section aims to give you an introduction to the features of each language generation. Note that although it seems as if each generation supersedes

the others, the ‘older’ languages are still very much in evidence. You will find many programs written in C and COBOL, and engineers frequently use FORTRAN.

- The first generation of languages was machine language. Instructions and addresses were numerical. These programs were linked to the machine they were developed on.
- The second generation allowed symbolic instructions and addresses. The program was translated by an assembler. Languages of this generation include IBM, BAL, and VAX Macro. These languages were still dependent on the machine they were developed on.
- Third generation languages allowed the programmer to concentrate on the problem rather than the machine they were writing for. Other innovations included structured programming and database management systems. 3GL languages include FORTRAN, COBOL, Pascal, Ada, C, and BASIC. All 3GL languages are much easier for the human being to understand.
- 4GL languages (fourth generation). These are known as non-procedural, they concentrate on what you want to do rather than how you are going to do it. 4GL languages include SQL, Postscript, and relational database orientated languages.
- 5GL (fifth generation). These languages did not appear until the 1990s, and have primarily been concerned with Artificial Intelligence and Fuzzy Logic. The programs that have been developed in these languages have explored Natural Language (making the computer seem to communicate like a human being).

Study Note

Structured programming was developed during the 1950s after Edgar Dijkstra’s insightful comments into the harmful nature of the GO TO statement. Dijkstra and others subsequently created a set of acceptable structures in programming that would enable development without GO TO statements. These structures produced programs that were easier to read by humans, easier to debug and easier to test. These structures have become some of the founding principles of modern programming methods.

Although the principles of structured programming have had a profound effect on the programming world, it was not until the 1970s that an actual language was created for teaching structured programming. Pascal was developed especially for this purpose, though it is much derided as a toy language, and appears to have never been used in commercial development. It appears that existing languages such as COBOL and FORTRAN were changed to accommodate Dijkstra's structures, or that programming included these structures through more indirect methods. Later generation languages such as C are fully-fledged structured programming languages; these are from the third generation and procedural, in that they are both written and executed step-by-step. C, in its turn, has formed the foundation for the object-oriented language C++.

The three structures allowed in structured programming are sequence, selection, and iteration. Structures are also thought of in terms of substitution and combination, i.e. structures can be substituted or combined with other structures as long as the result equals a sequential structure. Structured programming also pays attention to design and testing with emphasis on a top-down approach. The top-down approach uses modularity as a means to ensure that the program is both legible and manageable, and also that these modules can be tested as they are developed. This is beneficial as it ensures that all modules should be tested and that bugs can be found in the modules that have most recently been added or altered.

Structured programming also places emphasis on program documentation, which can be in the form of a chart or the structured coding/listing. This documentation allows for subsequent updating of modules, making these modules easier to locate in the program. Modularity also ensures greater opportunity for re-use of modules during development.

Study Note: Areas of change since 1993

There have been a number of changes in the programming community since the early 90s. Perhaps the most widely publicized of these has been the development of Java. Other areas of innovation have been Rapid Application Development Packages such as Visual

Basic and Borland Pascal. RAD applications have been heralded by their manufacturers as the solution to programming without programmers. Applications can be developed using a 'visual' or 'drag and drop' environment. Whilst it is the case that these programs do indeed speed up development, there are inherent problems with RAD that need to be considered during development. Emphasis on speed of development can mean that good design is neglected, and that the re-usability of components is not optimized. Time 'saved' during development, can lead to time lost after production as the application can become unmanageable and difficult to update.

Definition: Rapid Application Development

A programming system that enables programmers to quickly build working programs. In general RAD systems provide a number of tools to help build graphical user interfaces that would normally take a large development effort. Two of the most popular RAD systems for Windows are Visual Basic and Delphi.

Other areas of change include: -

- Web-based mark-up languages such as Cold Fusion and Active Server Pages that interface with databases to provide dynamic web content.
- A move towards distributed objects rather than the large mainframe applications of the past.
- The emergence of ActiveX and Java applets – mini programs that run independently so can be used to customize and create interactive web sites.
- Visual Basic being thoroughly integrated with Microsoft Office to customize programs.
- During the 90s networks, TCL (Tool Command Language), PERL (Practical Extraction and Report Language), CGI (Common Gateway Interface) etc have all become common in industry.
- ANSI (American National Standards Institute) standard for C++.

Definition: TCL

Short for Tool Command Language, and pronounced TCL, a powerful interpreted programming language developed by John Ousterhout. One of the main strengths of TCL is that it can be easily extended through the addition of custom TCL libraries. It is used for prototyping applications as well as for developing CGI scripts, though it is not as popular as PERL for the latter.

Definition: CGI

Abbreviation of Common Gateway Interface, a specification for transferring information between a World Wide Web server and a CGI program. A CGI program is any program designed to accept and return data that conforms to the CGI specification. The program could be written in any programming language including C, PERL , Java or Visual Basic.

Definition: PERL

Short for Practical Extraction and Report Language PERL is a programming language developed by Larry Wall especially designed for processing text. Because of its strong text processing abilities PERL has become one of the most popular languages for writing CGI scripts. PERL is an interpretive language that makes it easy to build and test simple programs.

Introduction to Object Oriented Concepts

Programmers who undertake structured analysis, design and programming think about data items and how to manipulate and organize them. Object orientation offers a different way to think about software systems. The philosophy of object orientation proclaims that applications can be built by envisioning objects that work together. In theory this should be an easier way to develop programs because we live in a world made up of objects, know how they relate to one another, and ought to be able to transfer our understanding of these things to the equivalent in software terms. Object orientation allows a developer to construct systems based on the idea of components as opposed to the structures that form

the basis of structured programming. This component based development lets objects be re-used or extended, educing development time dramatically.

Object oriented programming allows the following: -

- A system can be constructed from a set of objects just as a house is built from bricks, windows and doors (amongst other things).
- Adding new abilities to existing objects can expand a system a door for a house might be a door to a room or a cupboard door, this door could be extended to include garage doors.
- Creating new objects can expand a system; for example a house object may have stairs or a lift. No stairs might mean a bungalow, stairs would allow the house to have more than one floor, and a lift would allow a tall block of flats to be built. A house could be extended by including a conservatory.
- The bricks, windows and door objects that were designed for the first house can also be used to build other houses. This reduces the development time, as the existing design can be used to create other objects.

What is an Object - An introduction to Classes

The world around us is full of objects and modern software aims to imitate the real world. Just like things in the real world, objects have state and behaviour.

An object's state can be thought of the things it knows; a car might have an engine type, a make, model number and a colour. The car's behaviours are the things that it knows how to do, such as accelerating, braking, changing gear and turning the windscreen wipers on. The state for dog could be name, breed, colour, and hungry, and the dog can have behaviour such as snoring, barking, fetching and chasing cats.

An object can be:

1. Some physical thing in the 'real world'.
2. A representation of reality.

3. A tangible or visible thing.
4. A thing to which action or thought can be directed.
5. Passive - doing nothing until activated e.g. like a switch.
6. Active - continually monitoring until conditions change e.g. like a thermostat.

An object is never:

1. A value (e.g. name).
2. A process (e.g. sort).
3. Time (e.g. 5 minutes).

Classes

An object is an instance, or example, of a category or class. The house system that was described earlier is a class or category, and the types of housing, e.g. apartment, bungalow are instances of that class. In the real world, we are instances of the people class, and we may also be instances of other classes such as the brother class, mother class, and student class.

Software objects have state and behaviours; these form the structure of the object. The object's state is made up of items of data called attributes (or properties) which describe aspects of the object, whilst its behaviours are the operations that the object carries out.

Definition: Attribute

An attribute is a property of class. It describes a range of values the property may hold in objects (that is instances) of that class. Every object of the class has a specific value for every attribute.

Definition: Property

Characteristic of an object. In many programming languages the term property is used to describe attributes associated with a data structure.

Real world examples of object features are those that we might possess as a person. As instances of the person class, our attributes could be height, age, weight, nationality (note that the data for each of these attributes will be different in every case, because we as individuals are different) and our operations consist of eat, sleep, drink, go to college, do homework, play football, swim.

A developer wishing to create a system that was based on people-related information that could be used for a marketing purpose might look at the following attributes and operations for the person class. Attributes might consist of age, name, address, telephone number, email address, income and operations include takes holidays, reads magazines, drinks alcohol, goes shopping, and drives cars.

Classes have another advantage; they can be used as a template for creating other objects. The person class would give you a template for creating other instances of the person class. You have the template for creating a population.

The computer class may have a series of attributes such as brand name, model name, size of hard drive, processor speed. The computer class operations might consist of play latest games, play CD-ROMS, send email, surf web, and play music. The computer class can act as a template for creating new instances of the class, i.e. new objects. To put this more simply, the class can be thought of a mould for producing new objects.

This ability to create new objects easily is an important aspect of object oriented software development. An object in the real world has many attributes and behaviours, and it follows that the more features that you include with your class the more it will reflect reality. For example, the games computer could include the following attributes in its specification: how many MBs memory, the type of graphics card, floppy disk and so forth. There are many other operations that the computer could perform from desktop publishing to recording music.

Classification

Objects that share similarities and properties can be grouped together. Classification looks at the shared attributes and behaviours to create or describe classes of objects. A biologist studying a group of animals such as lizards, crocodiles, chameleons, turtles may decide that these belong to a class known as reptiles. These reptiles share behaviour such as cold-bloodedness and laying eggs and also data such as how big they are and what colour they are.

Classes are defined by specifying data or attributes and behaviour (operations or methods). The data for an object can also be referred to as its variables, and its operations or methods are known as its responsibilities. There are a number of different terms for responsibilities, and they may be referred to as operations, methods or functions. The reptiles class defined the data and responsibilities for the class as follows:

Reptiles Class	
Data	colour
	size
Responsibilities	Laying eggs
	Rearing young ones

D**efinition: Variable**

A unit of storage that can be modified during program execution usually by assignment or read operations. A variable is generally denoted by an identifier or by a name. In logic a variable is a name that can stand for any of an infinite set of values.

Responsibilities and data can be both stored in the class as a whole or for each instance of a class. The responsibilities or operations for the reptiles class could include the ability to grow. An instance of the reptiles class could grow to a particular size. Some instances of the reptiles class would be able to grow scales or be able to swim. The ability to move would be a class method, whilst the ability to swim would be an instance method.

Definition: Data

Information, in any form, on which computer programs operate. The distinction between a program (instructions) and data is a fundamental one in computing. It is in this fundamental sense that the word is used in terms such as data bus, data cartridge and data protection. Data can be distinguished from graphics, text, and speech. Data is distinguished by the fact that it is organized in a structured, repetitive, and often compressed way. This definition of the term data relates closely with database, data independence, data model and data processing.

Definition: Operations or methods

Operations or methods in a programming language. Whatever is carried out by an operator or, more generally, anything that can take place within a program: a declaration, an assignment, a selection, a loop, the call of a function and so on.

Different Sorts of Classes

There are two sorts of class that can be created, the abstract class and the concrete class. An abstract class may have one or more subclasses, but never an instance. In other words, an abstract class may be inherited by another class, but cannot become an instance of that class. A reptile is an abstract class, its data and responsibilities can be inherited, but a reptile is not an animal in its own right. A concrete class, however, is a class that can have one or more subclasses and/or instances. Instances of the reptile class such as lizard and crocodiles are concrete classes. They may have subclasses such as iguana and occur a number of times – have many instances.

Encapsulation and Abstraction

Information hiding is an important concept in object-oriented systems, and takes place in two important ways. Firstly, information about objects is hidden from other objects by the system itself, this is known as encapsulation. Secondly, the developer can choose to hide information about a class or object to ‘streamline’ that object to suit their needs. This is

known as abstraction.

Encapsulation

Encapsulation can also be referred to as black box technology. Black box is usually used to refer to technology that shields the user from its mechanics or technological workings. Most modern electronic appliances hide their technological processes from users. Appliances such as televisions, radios and so forth hide how they show television and radio programmes, and all the user sees is those processes being carried out. Most television viewers or radio listeners are not interested in how their equipment works, merely that it does.

With encapsulation an object bundles together its attributes (data) and its operations so that they are inseparable, they are part of the object. This information is hidden, hence information hiding, so that only the object knows about them and can act on them. For instance, a computer knows what processor it is using and how efficiently it is running, but there is no direct way to find out. Another example might be that human beings know how hungry they are and how to eat, but there is no direct way to find out how hungry that person is or how their digestion system works. To find any of these things out, the object must be sent a message, just as a human being would be asked whether or not they are hungry.

Encapsulation offers a distinct advantage to developers, as it reduces the potential for error. A broken object can affect the whole system, and if the software engineer needs to mend that object it may mean that the object can be repaired individually rather than repairing the whole system. In the real world, a television is usually linked to a video recorder, but they hide their operations from one another. If the television breaks down, it is only the television that needs repairing or replacing rather than the television, video and any other devices that might be attached. Encapsulation and information hiding mean that only an object can manipulate its data through using its own methods or operations. This prevents other objects from accessing its insides, and prevents accidents. A human

digestive system knows about its data and operations, but does not make these available to the outside world. This prevents accidents happening to an important object in the human body.

Although an object hides its operations from the outside world, it does allow a means of communicating with the outside world. This means of communicating is known as an interface. Interfaces are a feature of modern household items as well as encapsulation. Televisions and videos have remote controls to change channel and alter volume, microwaves have buttons or dials to set temperature and cooking duration, washing machines have dials to choose washing cycle. Computers communicate with users through an interface, for instance operating systems and other software packages that are being used.

Abstraction

Abstraction enables the developer to re-use a class, and filter out operations and attributes from that class that are superfluous to needs. An object may have a long list of associated features that are not always relevant to the system that the developer wishes to create. For instance, the marketing information class might provide extensive details of consumer lifestyles. A developer might decide to produce a system to sell information about pet-owning consumers to pet food manufacturers. The developer could choose to filter out operations such as the consumer's favourite restaurants and hobbies, which newspapers they read and how many visits they make to the cinema. The pet food manufacturer may only be interested in where pet owners live, where they shop, do they travel by car, how many pets of what type and what brand of pet food they buy. The software developer may also need to create software that tracks the number of consumer profiles in each area in order to find which areas are under-represented. In this case, it may be the case that the detailed information recorded for each profile may be more than is needed for the system. Consumer spending habits could be left out, leaving just the basic demographic information for each consumer. This consumer profile can be filtered in any way to suit the software developer's needs, and whatever the developer is left with after this process

is an abstraction of the consumer profile.

Messages and Operations

Objects communicate with each other by sending messages to one another. An object sends a message to another object telling it to perform an operation. This works in the same way as remote controls that send messages to objects such as televisions to tell them to perform operations such as change channel. The message itself only tells the object to perform the operation, not how to perform it. The object itself knows how to perform this operation from the class it belongs to.

When an object receives a message from another object, it activates a method or operation. These messages are also known as requests or function calls. The television remote control sends a message to the television to perform an operation or activate a method such as change channel. This process of transmitting a message from the Message Sender to the Message Receiver is called message passing. The Message Sender does not know how the Message Receiver carries out this method as this information is hidden, hence the term information hiding.

Messages may also contain arguments (parameters) to clarify the request. The television remote control will tell the television to change to a particular channel, to reduce the volume by one unit. A microwave can be asked to cook for a certain number of minutes, and a video to turn itself on at a particular time.

Methods

An object's methods are not necessarily available to all other objects in the system. Some methods can be accessed by outside objects and these are known as public methods. Other methods are only available by methods within the object itself, and these are known as private methods. In the real world, a computer would have some public methods that enable a human to interact with the system, such as open program and change screensaver. There are however, other methods that are available only to the computer

itself, such as accessing the RAM.

Relationships

There are a number of relationships that objects can have with each other. Principal among these is inheritance and association.

Inheritance

Classes are categories of objects, and an object is therefore an instance of a particular class. In providing a template for objects, a class also provides characteristics that are inherited by objects. A television object therefore inherits all the attributes and operations of the television class. Not only can an object inherit from its class, a class may also inherit from another class. This process is called inheritance. For example, a rice cooker, toaster, microwave and electric kettle are all classes in their own right, but each class is also a member of the kitchen appliances class. The kitchen appliance class is the parent or superclass of all the others, and these other classes inherit the characteristics of the kitchen appliance class; e.g., switch on, switch off, and buttons to turn the appliances operations on and off. The rice cooker, toaster, microwave and electric kettle are all therefore subclasses of the kitchen appliance class.

Superclasses can also be thought of as a form of taxonomy, in the real world mammals share certain inherited characteristics such as bearing live young, being warm blooded and feeding milk to their offspring. Subclasses of the mammals superclass inherit these properties, as well as their own particular characteristics. For instance, whales have fins and can swim underwater whereas cats have four legs and fur and usually remain on land.

Definition: Taxonomy

In a broad sense, the science of classification, but more strictly the classification of living and extinct organisms, i.e. biological classification. The term is derived from the Greek taxis.

Exercise

Consider the mammals superclass, and define some subclasses within that group, for instance, dogs and cats. What characteristics would these subclasses inherit?

Association

Objects are often associated with each other, in the same way that people might be associated with each other in daily life. These object relationships can either be one-way or they can be two-way. Objects can also have relationships that are either one-to-one, they may be one-to-many or alternatively an object can have different relationships with the same object. These associations are known as multiplicity.

A one-way or unidirectional relationship would exist between a human being and their television set. The human being can tell the television what operations to perform, but the television has no means to tell the human to perform operations.

A two way or bi-directional relationship exists where two people are married. A person might be your friend and also your boss, that person is in a ‘is a friend of’ association as well as a ‘is the boss of’ association. A dog can be a pet as well as a guard dog, the dog is therefore in a ‘is the pet of’ association as well as a ‘is a guard dog for’ association.

Classes can also associate with more than one class. A person can travel by boat and by aero plane. The person class is therefore associated with both the boat and the aero plane class.

Multiplicity

Multiplicity is a term that indicates the number of objects that are associated with one particular class. These associations can be one-to-one or one-to-many. For instance, a wife might have one husband; they have a one-to-one association. A mother, however, may have many children and she has a one-to-many association with those children. Multiplicity is very common in the real world; human beings walk on 2 legs (a one-to-

two association), dogs walk on 4 legs (a one-to-four association), spiders walk on 8 and insects on 6.

Polymorphism

Often developers will find that operations can have the same name even though they are associated with different objects. Polymorphism allows developers to re-use terminology and to allow it have more than one meaning. This can prove useful for a number of reasons. System modellers can talk to clients using terms that are familiar to them, and maintain client's own terminology. Some operations fall naturally into certain terminology such as open and close, and it would be preferable to use words that have an obvious meaning. The ability to allow more than one meaning for each operation means that the developer can maintain terminologies without having to invent a new unique word every time a similar operation occurs.

Polymorphism means that each object understands how it is supposed to perform an operation even though it may have the same name as another object's operation. For instance, just as a human understands how different objects in the real world open, so each class in object orientation understands how that operation occurs. A human being performs the operation 'open' on many different items in every day life, from doors to boxes, books, bank accounts and conversations. These are all different types of operation, although they share the same name. Each class would understand how it is supposed to perform the operation 'open' in its own right.

The term true polymorphism means that objects may share closely related method names. These polymorphic methods perform the same operation, but perhaps in a different way. Usually this stems from the object's inheritance as the different class methods may be related by their superclass.